**מבחן בקורס:** עקרונות שפות תכנות, 202-1-2051
**מועד:** ב
**תאריך:** 5/8/2025
**שמות המרצים:** מני אדלר, ירון גונן, יובל פינטר
**מיועד לתלמידי:** מדעי המחשב והנדסת תוכנה
**משך המבחן:** שלוש שעות
**חומר עזר:** אסור

**הנחיות כלליות:**

- יש לענות על כל השאלות בגיליון התשובות. מומלץ לא לחרוג מן המקום המוקצה.
- אם אינכם יודעים את התשובה, ניתן לכתוב 'לא יודע' על סעיף שלם ולקבל 20% מהניקוד.

| | | |
|---|---|---|
| **שאלה 1:** | תחביר וסמנטיקה _____ | נק 30 |
| **שאלה 2:** | מערכת טיפוסים _____ | נק 20 |
| **שאלה 3:** | תכנות פונקציונאלי, CPS, רשימות עצלות | נק 30 |
| **שאלה 4:** | תכנות לוגי _____ | נק 20 |
| **סה"כ** | _____ | נק 100 |

**בהצלחה!**

---

## שאלה 1: תחביר וסמנטיקה [30 נקודות]

**א.** [5 נקודות]

סמנו את הכתובות הלקסיקאליות בביטוי הבא:

```
(lambda (x y)
  (
    (lambda (y z) (+ y z x a))
    x  y  z
  )
)
```

**ב.** השפה L20 מוגדרת כשפה L2 אך ללא define

```
<program> ::= (L20 <cexp>+)              / Program(exps:List(cexp))
<var-ref> ::= <identifier>              / VarRef(var:string)
<var-decl> ::= <identifier>   / VarDecl(var:string)
<cexp> ::= <number>                     / NumExp(val:number)
        |  <boolean>                    / BoolExp(val:boolean)
        |  ( lambda ( <var-decl>* ) <cexp>+ ) /
                                ProcExp(args:VarDecl[],body:CExp[]))
        |  ( if <cexp> <cexp> <cexp> ) / IfExp(test: CExp,
        |                                      then: CExp,
        |                                      alt: CExp)
        |  ( <cexp> <cexp>* )              / AppExp(operator:CExp,
        |                                          operands:CExp[]))
<prim-op>  ::= + | - | * | / | < | > | = | not |  eq?
<num-exp>  ::= a number token
<bool-exp> ::= #t | #f
```

הגדירו באופן מפורט (אין צורך בקוד) אלגוריתם להמרת תוכנית ב-L2 לתוכנית ב-L20
[5 נקודות]

**ג.** ציירו דיאגרמת סביבות לכל אחת משתי התוכניות הבאות ב-L2 (בשפה זו, כזכור, אין עדיין סביבה התומכת ברקורסיות)

```
(L2

   (define mult
      (lambda (x n)
         (if (= n 1)
             x
             (+ x (mult x (- n 1))))
```

```
      )
   )

  (mult 3 2)

)
```

```
(L2

   (define mult
      (lambda (x n mult)
         (if (= n 1)
             x
         (+ x (mult x (- n 1) mult)))
      )
   )

  (mult 3 2 mult)

)
```

[10 נקודות]

**ד.** עדכנו את אלגוריתם ההמרה מ-L2 ל-L20, כך שהוא יתמוך גם בביצוע רקורסיות.
[5 נקודות]

**ה.** האם ניתן לבצע את האלגוריתם ברמת התחביר, ללא צורך בחישוב ביטויים, או שהוא דורש חישוב של ביטויים?
[5 נקודות]

## שאלה 2: טיפוסים [20 נקודות]

נרחיב את שפת הטיפוסים L5 לשפה L55, שתתמוך בהצהרה מפורשת על תופעות לוואי בחתימת הפונקציה.
ההרחבה כוללת את הכלל הבא: **פונקציה שמייצרת תופעת לוואי חייבת לציין זאת בטיפוסה בעת ההגדרה שלה, ולהפך** – כל הצהרה על תופעת לוואי בטיפוס מחייבת קיום תופעת לוואי בגוף הפונקציה. שימו לב שבהיעדר תופעות לוואי בגוף הפונקציה, לא תופיע הצהרה בחתימה.
לצורך השאלה נסתפק בתופעת הלוואי "הדפסה", כלומר פקודת `display`, ונניח שהיא היחידה הקיימת בפונקציות. על בודק הטיפוסים לוודא שכל פונקציית משתמש שיש בה שורת `display` מתויגת בהתאם בטיפוס החוזר שלה, ולהפך. הצהרה על תופעת לוואי נעשית באמצעות הוספת המילה `prints` לפני ערך ההחזרה של הפונקציה, כלומר: `prints ret-val`. לדוגמא הפונקציה `add` שמחברת שני מספרים, מדפיסה את הסכום ומחזירה אותו, תיראה כך:

```
(define (add : (number * number -> prints number))
    (lambda ((x : number) (y : number)) : prints number
        (let (((z : number) (+ x y)))
            (display z)
            z)))
```

המילה המיוחדת `prints` מתפרשת (בשפה טבעית) כ"הפונקציה מדפיסה *משהו*", כלומר אין חובה שמה שמודפס הוא הערך החוזר (או משהו מאותו טיפוס). הקוד הבא תקין:

```
(define (add : (number * number -> prints number))
    (lambda ((x : number) (y : number)) : prints number
        (let (((z : number) (+ x y)))
            (display #t)
            z)))
```

את מבנה הנתונים של הביטוי החדש נגדיר כך:

```
export type PrintsTExp = { tag: "PrintsTExp" }; // and predicate and make...

export type ProcTExp = { tag: "ProcTExp"; paramTEs: TExp[]; printsTE:
TExp|null; returnTE: TExp; };
export const makeProcTExp = (paramTEs: TExp[], printsTE: TExp|null, returnTE:
TExp): ProcTExp =>
    ({tag: "ProcTExp", paramTEs: paramTEs, printsTE: printsTE|null, returnTE:
returnTE});
export const isProcTExp = (x: any): x is ProcTExp => x.tag === "ProcTExp";
export const procTExpComponents = (pt: ProcTExp): TExp[] =>
    [...pt.paramTEs, pt.printsTE, pt.returnTE];

export const parseTExp = (texp: Sexp): Result<TExp> =>
    ...
    (texp === "prints") ? makeOk(makePrintsTExp()) :
    ...
```

**א.** עדכנו את פונקציית זיהוי ביטויי הטיפוס הפונקציונליים ב-`TExp.ts` לתמיכה בצורה החדשה:
[5 נקודות]

```
/* contents of fail messages omitted for clarity, you can do the same */
const parseCompoundTExp = (texps: Sexp[]): Result<ProcTExp> => {
    const pos = texps.indexOf('->');
    return (pos === -1)  ? makeFailure(`...`) :
           (pos === 0) ? makeFailure(`...`) :
           (pos === texps.length - 1) ? makeFailure(`...`) :
           (texps.slice(pos + 1).indexOf('->') > -1) ? makeFailure(`...`) :
           bind(parseTupleTExp(texps.slice(0, pos)), (args: TExp[]) =>
               mapv(parseTExp(texps[pos + 1]), (returnTE: TExp) =>
                    makeProcTExp(args, returnTE)));
};
```

**ב.** עדכנו את קוד בדיקת הטיפוסים עבור ביטויי למבדה, `typeofProc`, כך שיאכוף את ההצהרה עם הדרישה העדכנית.
[7 נקודות]

העזרו בפיסות הקוד הממוסגרות, מתוך TExp.ts ומבדיקת הטיפוסים בהתאמה:

```
export type VoidTExp = { tag: "VoidTExp" };
export const makeVoidTExp = (): VoidTExp => ({tag: "VoidTExp"});
export const isVoidTExp = (x: any): x is VoidTExp => x.tag === "VoidTExp";
```

```
export const typeofPrim = (p: PrimOp): Result<TExp> =>
    ...
    (p.op === 'display') ? parseTE('(T -> void)') :
    ...
```

```
export const allTypeofExps = (exps: List<Exp>, tenv: TEnv): Result[]<TExp> =>
//@TODO

export const typeofProc = (proc: ProcExp, tenv: TEnv): Result<TExp> => {
    const argsTEs = map((vd) => vd.texp, proc.args);
    const extTEnv = makeExtendTEnv(map((vd) => vd.var, proc.args), argsTEs,
tenv);
    const constraint1 = bind(typeofExps(proc.body, extTEnv), (body: TExp) =>
                            checkEqualType(body, proc.returnTE, proc));
    return bind(constraint1, _ => makeOk(makeProcTExp(argsTEs,
proc.returnTE)));
};
```

**ג.** בסעיפים הקודמים התמקדנו במקרה שבו הצהרה על תופעת לוואי מחייבת את קיומה בפועל בגוף הפונקציה.
בסעיף זה נרחיב את המערכת על ידי הוספת תיוג חדש `maybe-prints` (בנוסף לתיוג הקיים `prints`),
אשר מציין שתופעת הלוואי היא אופציונלית: ייתכן שקיימת בגוף הפונקציה וייתכן שלא.
תארו בקווים כללים (ללא קוד) את השינויים הנדרשים בתחביר ובבדיקת הטיפוסים (המעודכנים מהסעיפים הקודמים) כדי לתמוך בשינוי הנתון.
[8 נקודות]

```
(define (add : (number * number -> maybe-prints number))
    (lambda ((x : number) (y : number)) : maybe-prints number
        (let (((z : number) (+ x y)))
            (if (> z 0)
                ((display #t)
                 z)
                z))))
```

## שאלה 3: תכנות פונקציונאלי, CPS, רשימות עצלות [30 נקודות]

**א.** הקיפו בעיגול נכון / לא נכון [2 נקודות לכל סעיף, סה״כ 10 נקודות]

א.1. אם ביטוי נמצא בעמדת זנב, אז הוא בהכרח הקריאה האחרונה בגוף הפונקציה.
נכון / לא נכון

א.2. עמדת זנב, **באינטרפרטר התומך באופטימיזציית זנב**, מבטיחה שקריאה לפונקציה לא תוסיף פריים חדש למחסנית הקריאות.
נכון / לא נכון

א.3. כתיבה של פונקציה בסגנון CPS מבטיחה שהקריאה תמיד תהיה בזנב.
נכון / לא נכון

א.4. הקריאה (f (g x)) נמצאת בעמדת זנב אם f היא הקריאה האחרונה.
נכון / לא נכון

א.5. עמדת זנב רלוונטית רק בשפות פונקציונליות.
נכון / לא נכון

**ב.** רשימות עצלות ו-CPS
[12 נקודות]
ב.1. [6 נקודות] הפונקציה take שימושית מאוד בעולם הרשימות העצלות. היא מקבל רשימה עצלה ומס׳ איברים, ומחשבת רשימה רגילה המכילה את מס׳ האיברים הראשונים מתוך הרשימה העצלה. להלן הקוד שלה:

```
;; [ Lzl[T] * Number -> List[T] ]
(define take
  (λ (lzl n)
     (if (or (= n 0) (empty-lzl? lzl))
         empty
         (cons (head lzl)
          (take (tail lzl) (- n 1))))))
```

כתבו גרסה בסגנון CPS עבור פונקציה זו:

```
;; [ Lzl[T1] * Number * [List[T1] -> T2] -> T2 ]
(define take$
  (λ (lzl n cont)
     (if (or (= n 0) (empty? lzl))
         (cont empty-lzl)
```

ב.2. [8 נקודות] הוכיחו כי שתי הפונקציות שקולות-CPS

ג. תכנות פונקציונלי ו- Currying פונקציונלי [6 נקודות]

השלימו את הקוד עבור הפונקציה make-cond-accumulator אשר מקבלת כפרמטרים פונקציית צבירה/מיזוג (למשל + או * ) וערך נייטרלי (0 או 1 בהתאמה למשל), ומחזירה פונקציה נוספת המקבלת פונקציית פרדיקט ורשימה ומבצעת את הצבירה רק על איברים העונים על התנאי בפרדיקט. ראו דוגמאות בחתימת הפונקציה:

```
;;[ [T1 * T2 -> T2] * T2 -> [[T1 -> Boolean] * List[T1] -> T2] ]
;; Examples:
;;  ((make-cond-accumulator + 0) (λ (x) (< x 5)) '(1 2 5 6 3))
;;  => 6
;;  ((make-cond-accumulator cons '()) (λ (x) (< x 5)) '(1 2 5 6 3))
;;  => '(1 2 3)
(define make-cond-accumulator
  (λ (combine neutral)
     (λ (pred lst)
     (cond
       ((null? lst) neutral)
```

## שאלה 4: תכנות לוגי [20 נקודות]

נגדיר עץ בינארי ע"פ החוק/יחס הבא:

```
binary_tree(void).
binary_tree(tree(Element,Left,Right)) :-
   binary_tree(Left), binary_tree(Right).
```

**א.** האם ניתן להגדיר עץ בינארי ללא שימוש בפנקטורים? נמקו.
[2 נקודות]

**ב.** ממשו את הפרוצדורה `tree_to_list` הנותנת ערך אמת לעץ ולרשימה, אם הרשימה כוללת את האלמנטים של העץ בסדר in_order (קודם האלמנטים של תת-העץ השמאלי, אחר כך האלמנט של העץ, ואחר כך האלמנטים של תת-העץ הימני). ניתן להשתמש ב-`append` שנלמדה בכיתה.

```
?-tree_to_list(tree(a,tree(b,void,void),tree(c,void,void)),L)
L = [b,a,c]

?-tree_to_list(tree(a,tree(b,tree(d,void,void),void),tree(c,void,tree
(e,void,void))),L)
L = [d,b,a,c,e]

% Signature: tree_to_list(Tree,List)/2
% Purpose: List is a list of the Elements in Tree (inorder)
```

[8 נקודות]

**ג.** ציירו את עץ ההוכחה עבור השאילתא הבאה, וציינו האם העץ הוא סופי או אינסופי והאם הוא עץ הצלחה או כישלון:

```
person(danny, location(bs)).    %1
person(dinna, location(tlv)).   %2
person(ruth, location(jrs)).    %3

course(spl, bgu, location(bs)).    %4
course(ppl, tau, location(tlv)).   %5
course(algo, heb, location(jrs)).  %6
```

```
distance(jrs, bs, long).    %7
distance(jrs, tlv, short).  %8
distance(jrs, jrs, short).  %9

available(Person, Course):-        %10
   person(Person, location(PCity)),
   course(Course, _Univ, location(CCity)),
   distance(PCity, CCity, short).


?-available(ruth,Course)
```

[10 נקודות]